Hermes Agents vs LangChain Agents

> Written against what this repo actually builds — a trade-compliance Researcher with real tools — and what the same system would look like in LangChain. Hermes claims cite the installed runtime (v0.18.2, MIT, Nous Research); LangChain claims verified against current docs, mid-2026. Token and reliability figures are measured on this repo, not estimated.

Hermes is a runtime you configure. LangChain is a library you assemble.

Hermes hands you a finished agent and asks what it should be allowed to do. LangChain hands you the parts and asks what you want to build. Both get you a tool-using agent quickly; they differ in everything *around* the agent.

THE THREE DIMENSIONS

	HERMES	LANGCHAIN
Architecture	A running process. One fixed think-act-observe loop (`AIAgent.run_conversation()`) serves the CLI, the web gateway, cron jobs and chat platforms alike. You choose what it can reach, not how it thinks. *Less to build and less to get wrong — but no way to express a flow the loop does not already support.*	A library you import. `create_agent` is a shortcut over LangGraph; underneath you draw the graph yourself — nodes, branches, loops, pause-for-a-human. *Expresses anything you can draw. You maintain all of it.*
Tool definition	A small MCP server named in `config.yaml` , or a plugin taking a plain Python function. The description the model reads is generated from the function itself, so the two cannot drift apart. Hermes wraps it in injection scanning, a malware check and a stripped environment. *A tool lives outside the framework and costs nothing to anyone who does not switch it on.*	The `@tool` decorator, attached per agent with `.bind_tools()` . Same idea — the docstring teaches the model when to call it. MCP works too, via `langchain-mcp-adapters` , as an extra package rather than a config key. *The quickest possible way to add a tool. Any sandboxing is yours to add.*
State management	Four things, all included, no extra service: `MEMORY.md` facts pasted into the prompt each session; every past conversation searchable in SQLite (`session_search` , no model call involved); reusable Skills; and one optional hosted provider behind a single config key. *Memory for free, on its terms — exact-word search, not meaning-based, unless you plug in a provider.*	Two systems you choose and wire: a **checkpointer** for the current conversation (`SqliteSaver` / `PostgresSaver`), and `langgraph.store` for anything longer-lived — usually reached through a tool you write yourself. *Nothing by default, everything possible — including meaning-based search. More wiring, more reach.*

WHAT THAT MEANS IN PRACTICE

	HERMES	LANGCHAIN
The agent loop	included	included
Conversations that survive a restart	included	wire a checkpointer
Memory that outlives the session	included	wire a store + a tool to write with
A second agent	one config line (`delegate_task`)	build the graph
Swap the model	a four-line YAML file	a code change + a pip install
Web API with streaming	included	write a server
Where the agent lives	it **is** the process	sits inside your app

For a two-tool demo, LangChain is less work. This exercise asked for memory that outlives the session, a second agent and a model swap — which is exactly where the columns flip.

WHICH ONE FOR WHICH JOB

IF YOU ARE BUILDING	REACH FOR	BECAUSE
An internal assistant a team talks to all day — this compliance desk, an ops helper, a research aide	**Hermes**	It has to remember last week, run on your own hardware, and be reachable from Slack and a browser. All included; none of it is your code.
A scheduled agent — check something each morning, post the result to a channel	**Hermes**	Cron, messaging and sessions ship inside the same runtime. In LangChain each is a separate thing you host.
Anything that must stay on your own machines — regulated data, no third-party service	**Hermes**	MIT, fully local, no external dependency by design.
A feature inside a product you already have — a "summarise this ticket" button in your SaaS	**LangChain**	You need a library your app calls, not a second process running beside it.
Question-answering over your own documents	**LangChain**	Retrieval and vector stores are its home ground. Hermes does not model them at all.
A flow with approvals or branches — pause for a human, take a different path, loop until a check passes	**LangChain**	Hermes has one loop shape. If your process does not fit it, you cannot express it.

TWO THINGS WORTH SAYING OUT LOUD

Neither framework's value is the loop. Think-act-observe is about fifteen lines and you can write it yourself. The value is everything around it that you only discover you need once real users arrive — retries, trimming long conversations, saving sessions, searching them, deciding what is worth remembering, handing work to a second agent.

A framework's defaults are the real cost. Hermes switches on 49 tools by default, tuned for a general assistant. Cutting this agent down to the five it actually uses took the prompt from **21,373 to 6,694 tokens per call** — and tool-calling got *more* reliable, because a model choosing between five options picks better than one choosing between 49.

CHOOSING

> **Hermes** when the agent *is* the product, and its shape fits one loop with delegation.
>
> **LangChain** when the agent is one feature inside something bigger, or you need document retrieval or a flow that branches.
>
> **Neither** when you have two tools, one user and nothing to remember. That is the loop plus your own functions, and a framework will cost you more in defaults than it saves you in code.

Both are MIT, and neither is settled ground. LangChain has changed its agent API twice in about a year (`AgentExecutor` → `create_react_agent` → `create_agent`), and Hermes is pre-1.0 — building this found its documentation contradicting its own source in three places. **Pin your versions and read the source, whichever you pick.**

Evidence — the fifteen-line loop written out, what a tool description actually contains, the three ways into Hermes and why this repo took the third, and where open-weight models break — is in `hermes-vs-langchain-detail.md`.